159.102 Notes

Section 12 : Namespaces + argc and argv + Game of Life

Topic 1 : Namespaces

"A namespace in C++ is a declarative region that provides scope to the identifiers inside it, including classes, functions, variables, etc".

This definition of a namespace is rather complicated and I do not recommend that you try to remember it. What is important for advanced programming is to be sure about what it means to use namespaces and the reasons why we do this.

The main reason for namespaces is that in our quest for meaningful variable names, we often want to use the same name for different variables. Let us look at an example program:

```
#include <iostream>
using namespace std;

namespace mynamespace {
  int value;
}

int value;

int main() {
  value = 150;
  mynamespace::value = 202;
  cout << "The global value is " << value << endl;
  cout << "The value in the namespace is ";
  cout << mynamespace::value << endl;
}
```

Notes about this program:

1. We set up a namespace called `mynamespace`. Inside the namespace we can put any number of classes, variables and functions. In this example, we have one variable called `value`.
2. Note that the namespace is not a class. It is simply a collection of classes, variables and functions grouped together in one namespace.
3. There happens to be a global variable also called `value`. We use the global variable as usual.
4. When we want to use the variable from inside the namespace, we have to put the name of the namespace in front of the variable name, like this: `mynamespace::value`. Just like we do when we are defining functions that are methods in a class.
5. Using the namespace system, we can have two variables with the same name in the same program. We just need to clearly inform the system which variable we are using.

Now you may be thinking – this is all crazy. There is no need to do this – just make sure you give every variable a unique name. This is true for simple programs, such as the example above. We could rewrite this example by leaving out the namespace and having two variables, `value1` and `value2`.

However, things get more complicated when we start using libraries. Most libraries do not just contain classes and functions, they often also contain variables. When you are using a library, you often don't know what names they have used for their variables. You may inadvertently use the same name for a

variable or function in your own program and this could cause a conflict. In short, most libraries in C++ are namespaces – a collection of related classes, variables and functions.

An example of a namespace - `fstream`

When we use text files, we usually use the library called fstream. And we usually have code in our program that looks like this – note that this is not a complete program:

```
#include <fstream>
fstream datafile;
datafile.open(filename.c_str(), fstream::out);
```

Notice that the second parameter to the method called `open` is a variable called `out`. But it is a variable that has been set up as part of the fstream namespace inside the fstream library. Therefore we have to use the full name for the variable which is `fstream::out`. This allows us to use our own variable called `out` (if we want to). In particular, if we did happen to create our own variable called `out`, we do not inadvertently modify the special variable called `out` that is inside the fstream namespace. The purpose of that special variable is to inform the system that data will be going out of the program and into the file.

Using a namespace

And so we can finally get around to answering the question that some people have been asking since the start of 159.101 which is "why do I have to put this line into my program: `using namespace std;`?". Unfortunately, we had to wait until we had covered libraries and classes before we could fully answer this question.

We have discussed the use of namespaces above and we notice that names start to get rather long, for example we have `mynamespace::value` and we have `fstream::out`. This started to annoy some programmers (who love finding ways to reduce their typing) and so they came up with an alternative plan. Instead of having to place the name of the namespace before every function or variable, they would simply state `using namespace..` at the top of the program. This means that the system assumes that every "special" variable or function in the program belongs to the stated namespace. In this context, "special" means "not otherwise defined".

A good example is the std namespace. std is short for "standard" and it is a huge namespace with many functions and variables. The reason we put `using namespace std;` at the top of our programs is that we don't want to explicitly have to put std:: in front of many functions and variables.

Here is an example of a program that does ***NOT*** have that line in it:

```
#include <iostream>

int a, b;

int main() {
  std::cout << "Enter a number ";
  std::cin >> a;
  b = a + 5;
  std::cout << "Result is " << b << std::endl;
}
```

As you can see, we now have to include `std::` all over the place, including for the variable `std::endl`.

In some cases this gets particularly bad, for example, here is a line of code to run a timer in your program (if you want to see how long your program takes to run):

auto elapsed = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin);

That line alone explains why we use `using namespace std;` in our programs. By the way, if you are interested in setting up a timer in your programs, find out more here: https://levelup.gitconnected.com/8-ways-to-measure-execution-time-in-c-c-48634458d0f9

Best practice – do *NOT* put `using namespace std` into your programs

Yes, we have always put `using namespace std;` into our programs. It makes it easier to write your program and, for simple programs with no other namespaces, it works very well. However, as your programs get more advanced and you use more libraries and more namespaces, this instruction can cause a problem.

Note that the using namespace instruction can be used for any namespace. For example:

```
include <iostream>
using namespace std;

namespace mynamespace {
  int value;
}

using namespace mynamespace;

int other;

int main() {
  other = 150;
  value = 202;
  cout << "The global value is " << other << endl;
  cout << "The value in the namespace is ";
  cout << value << endl;
}
```

Note we have included the line `using namespace mynamespace;` which means we can refer to `value` instead of `mynamespace::value`

However, this immediately causes a problem. In the original version of this program (above) there was a global variable called `value`. In that original version of the program there was a clear distinction between `value` and `mynamespace::value`. In this latest version, that clear distinction has been removed.

Expert programmers recommend **NOT** including the line `using namespace mynamespace;` and being very clear about which variables (or functions) belong to which namespace by using the full name. You do NOT have to stop including `using namespace mynamespace;` You just need to be aware of problems that can arise with names and namespaces.

Topic 2 : Systems Programming

A systems program (or batch program) is a program that is designed to run within the system without interaction with a human user. For example, your computer may have a virus checker. This type of program runs and may produce a log file or may display a warning on the screen but it does not sit around waiting for a human user to enter some data.

On the other hand, an application program (or interactive program) is a program that uses the kind of input/output we are familiar with (such as cin and cout). It waits for a human user to enter data and then reacts in some way, and then often goes back to waiting for more data to be entered. For example, a search program on a mobile device waits until you request it to find the phone number of a particular person.

Output from a system program does not have to be that different from the output from an application program. A systems program can display information on the screen (using cout) or send output to a text file (often known as a log or a report when it is from a system program).

The most important difference between a systems program and an application program is the input. Systems programs do not use cin in the usual way, simply because cin will cause a program to sit doing nothing until data has been entered. One way to manage the input to a systems program is to read all input from a text file. Such a file is sometimes known as a *parameter file*.

Challenge – write a program to do something useful (e.g. a calculation) and do not accept any input from the screen. Read all the required input data from a text file. This input file has to be modified in order to make the program work with different input data.

Another way to manage input data for a system program is to use ***command line arguments***.

Command line arguments in C++

The command prompt and the use of system commands were covered in Section 9 of the notes. Recall that you can run your own programs from the command prompt as long as you have already created the executable version (the .exe file) of your program.

For example, if I have a program called prog1.cpp, I use Atom to create the executable file called prog1.exe and then it is ready to run. I can double click on the .exe file or I can go into the command prompt window (also known as the console or terminal window). Remember to navigate to the correct directory (folder) in the command prompt window. I enter the name of the program and it will run from the command prompt.

When running from the command prompt I also have the option of including command line arguments along with the name of the program.

Using `argc` and `argv` in C++

The first step to using command line arguments is to rewrite `main` to include the command line arguments. The first line of `main` will now look like this:

```
int main(int argc, char* argv[]) {
```

Important notes about main:

- main is the first function that is run when starting to execute a program

- main is a function – it returns an integer and can have (optional) two parameters
- the return value of main can be used to return an error code to the system (we don't do this)
- the two parameters are `argc` and `argv` – discussed in more detail below

The first parameter is `argc` – this stands for "argument count", a positive integer that is the number of command line arguments. Note that argc includes the name of the file. E.g. if you had 2 input values to your program then argc would be 3 and the three arguments would be (a) the name of your program, (b) the first input value, and (c) the second input value.

The second parameter is argv – this stands for "argument vector", an array of C strings (data type is char *). These strings hold the command line arguments. The first string holds the name of the file that is being executed.

Let us work our way through an example which is a simple program to add two numbers.

Program 1 : a simple program that does not use command line arguments

Program 1 is how we would normally write the program without bothering about argc and argv.

```
#include <iostream>
using namespace std;

float a, b , result;

int main() {
  cout << "Enter the first value ";
  cin >> a;
  cout << "Enter the second value ";
  cin >> b;
  result = a + b;
  cout << "a + b = " << result << endl;
}
```

Program 2 : modify program 1 to use argc and argv

Now we introduce argc and argv. Remember that the whole point of the exercise is to create a *systems program*. i.e. to remove the input so no human interaction is needed. We want to set the program up in such a way that you enter the name (to execute it)

```
#include <iostream>
#include <cstdlib>    // to use exit
using namespace std;

float strtofloat(string str);

string temp;
float a, b , result;

int main(int argc, char* argv[]) {
  if (argc != 3) {
    cout << "ERROR 12: you must supply two numbers";
    exit(12);
```

```
  }
  // ignore argv[0] - this is the name of the program
  temp = argv[1];
  a = strtofloat(temp);
  temp = argv[2];
  b = strtofloat(temp);
  result = a + b;
  cout << "a + b = " << result << endl;
}

float strtofloat(string str) {
  float value, fraction;
  int len, i;
  i = 0;  // very important to set i to zero
  value = 0.0;
  len = str.length();
  while (str[i] != '.') {
    value = value * 10 + str[i] - 48;
    i++;
    if (i == len) { return value; }  // if there is no dot
  }
  i++;  // skip over the dot
  fraction = 0.1;
  while (i < len) {
    value = value + (str[i] - 48) * fraction;
    fraction = fraction / 10.0;
    i++;
  }
  return value;
}
```

Important notes about program 2:

- it looks enormously longer than program 1 but most of it is the function strtofloat – focus on main to see the differences between program 1 and program 2
- note the new parameters for main
- note that all input (cin instructions) have gone – this is what we want for a systems program
- all prompts have also gone – you need to know what you must supply to a systems program
- we check to see that argc is equal to three because we know there should be three command line arguments – the name of the program and two numbers
- IMPORTANT – load argv[x] into a C++ string before using it. This is because argv consists of C strings and they may not work correctly in a C++ program
- because argv consists of strings, we need to convert strings to numbers in order to continue. I am sure you could find a "strtofloat" function in a library – I prefer to write my own functions

In order to run program 2, you need to do the following:
- enter your program into Atom and build it with no errors – this will create the .exe file
- go the console/terminal/command prompt window
- navigate to the correct directory (folder) where your .exe file has been saved
- enter the name of your program followed by two values, for example: `prog1 4 2.85`

(in this example, the program should run and produce the result 6.85)

Topic 3 : The Game of Life

This is ***not*** part of our course. Feel free to stop reading here and go no further.

The Game of Life is included here because:

a) it is a famous study in Computer Science that led to many other research topics
b) expert programmers should be aware of Agent-Based Systems or Multi-Agent Systems
c) you know enough C++ to be able to write the Game of Life program

The Game of Life is a *cellular automaton* devised by John Conway in 1970. It formed the basis for many further ideas in the area of Agent-Based Modelling (ABM) also known as Agent-Based Systems or Multi-Agent Systems. This is a fascinating area of study for programmers.

Multi-Agent Systems have been used in a number of different areas including finding the best way to exit a building on fire, simulations of ecological systems, and battle scenes in the Lord of the Rings movies.

Further reading about the Game of Life:
https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life

Further reading about Multi-Agent Systems:
https://en.wikipedia.org/wiki/Multi-agent_system

Further reading about simulations of buildings on fire:
https://www.xfire.co.nz/services/computer-modelling/

Read about the MASSIVE system developed in NZ:
https://en.wikipedia.org/wiki/MASSIVE_(software)

The code for the Game of Life is available on Stream under Week 12 called GameOfLife.cpp